



Kurulum ve İlk Cluster

Kubernetes ve Konteyner Dünyası



Bu Konuda

- Öğrenme ortamları: kind ve minikube
- `kubectl` kurulumu ve sürüm uyumluluğu
- Kubeconfig ve context
- İlk cluster: oluşturma, doğrulama, silme
- Temel komutlar
- Çıktı biçimleri

Öğrenme Ortamları





Nerede Pratik Yapılır?

Kubernetes'i yerelde çalıştırmak, öğrenmek için güvenli ve düşük maliyetli bir ortam sağlar; cluster'lar kolayca kurulup silinebilir.^[1]

- **kind:** Docker konteynerlerini node olarak kullanan hafif cluster
- **minikube:** tek node'lu yerel cluster
- **Çevrimiçi playground:** tarayıcıda anında deneme ortamı



kind ve minikube

	kind	minikube
Çalışma prensibi	Docker konteynerleri node olarak (kubeadm + kubelet)	Yerel tek node
Öncelik	Kubernetes'i test etmek; öğrenme için de uygun	Öğrenme ve geliştirme
Node sayısı	Tek veya çoklu node	Tek node
Platform	Linux, macOS, Windows	Linux, macOS, Windows

kubectl ve Kubeconfig





kubectl: Kurulum ve Sürüm

- `kubectl`, Kubernetes cluster'larıyla konuşmak için kullanılan komut satırı aracı; Kubernetes API'sini kullanır^[2]
- Kurulum: binary veya paket yöneticisi ile yapılır
- kubectl sürümü, cluster'dan **en fazla bir minor sürüm** farklı olmalı^[6]

```
kubectl version --client      # yalnızca kubectl sürümü  
kubectl version              # Client ve Server sürümleri
```

- Örnek: v1.36 client, v1.35 ile v1.37 arası control plane'lerle uyumlu
- Tam sürüm farkı (version skew) politikası
→ [Upgrade ve Sürüm Yönetimi](#)



Kubeconfig: Yapı ve KUBECONFIG

- kubectl, bir cluster'a nasıl bağlanacağını **kubeconfig dosyasından** öğrenir; varsayılan yol `$HOME/.kube/config` 'tir^[5]
- Dosya dört ana bölümden oluşur: `clusters`, `users`, `contexts`, `current-context`

```
apiVersion: v1
kind: Config
current-context: kind-kind
clusters:
- name: kind-kind
  cluster:
    server: https://127.0.0.1:port
contexts:
- name: kind-kind
  context:
    cluster: kind-kind
    user: kind-kind
```

- **KUBECONFIG** ortam değişkeni birden çok kubeconfig'i **birleştirir**; `--kubeconfig` bayrağı yalnızca belirtilen dosyayı kullanır



Context: Cluster + User + Namespace

- **Context**, bir kubeconfig içinde cluster, namespace ve user'ı **tek bir ad altında gruplar** ^[5]
→ [Kubernetes API ve Manifest Okuryazarlığı](#)
- kubectl varsayılan olarak **current-context**'teki bilgileri kullanır

```
kubectl config view          # birleştirilmiş yapılandırmayı gösterir
kubectl config current-context # etkin context'i gösterir
kubectl config use-context kind-kind # context değiştirir
kubectl cluster-info         # cluster uç noktasını doğrular
```

- Çoklu cluster'da **use-context** ile hızlı geçiş yapılır

İlk Cluster





İlk Cluster: Oluştur, Doğrula, Sil Laboratuvar

kind ile tek komutla cluster kurulur; varsayılan context adı `kind-kind` 'dir.^[7]

```
kind create cluster      # cluster oluşturur
kind get clusters        # oluşturulan cluster'ları listeler
kubectl get nodes        # node'ları ve durumlarını doğrular
kind delete cluster      # cluster'ı siler (idempotent)
```

minikube da benzer bir akış izler:^[8]

```
minikube start           # driver seçip cluster başlatır
minikube stop            # cluster'ı durdurur
minikube delete --all    # tüm cluster'ları siler
```

Temel Komutlar





kubectl Sözdizimi

Tüm kubectl komutları aynı şablonu izler: ^[2]

```
kubectl [command] [TYPE] [NAME] [flags]
```

- **command:** yapılacak işlem (ör. `create`, `get`, `describe`, `delete`)
- **TYPE:** nesne türü; tekil, çoğul ve kısaltma aynı sonucu verir (ör. `pod`, `Pods`, `po`)
- **NAME:** nesne adı; atlanırsa o türdeki tüm nesneler listelenir
- **flags:** seçenekler (ör. `-o` çıktı biçimi, `--context` context seçimi)

```
kubectl get pod pod1  
kubectl get pods pod1  
kubectl get po pod1
```



get ve describe

- `kubectl get` özet durum verir: nesne türüne göre temel sütunlar listelenir^[3]
- `kubectl describe` ayrıntılı durum ve **olayları ("event")** gösterir; hata incelemede ilk adım olarak kullanılır

```
kubectl get nodes           # node'ların durumunu listeler
kubectl get pods            # Pod'ları listeler
kubectl get pods -o wide    # ek sütunlarla listeler
kubectl describe pod <pod-adı>
kubectl describe node <node-adı>
```



create ve run

- `kubectl create` nesne oluşturur; `kubectl run` tek bir Pod başlatır^[3]

```
kubectl create deployment nginx --image=nginx  
kubectl run nginx --image=nginx
```

- Bu **imperative** komutlar geliştirmede hızlı; gerçek ortamda **declarative** dosyalar tercih edilir
→ [Kubernetes API ve Manifest Okuryazarlığı](#)



delete, logs ve exec

- Nesneleri silme ve çalışan konteynerlerle etkileşim: [\[3\]](#)

```
kubectl delete pod unwanted --now      # bekletmeden siler
kubectl delete pod,service baz foo      # birden çok tür birlikte
kubectl logs my-pod                     # konteyner çıktısı (stdout/stderr)
kubectl logs -f my-pod                   # canlı log akışı
kubectl exec -it my-pod -- /bin/sh      # çalışan konteynerde kabuk
```

- `kubectl logs` yalnızca çıktıyı okur; dosyaya yazılan loglar için konteyner içine girmek gerekir

Çıktı Biçimleri





Çıktı Biçimleri

- `-o` (veya `--output`) bayrağı çıktıyı makine tarafından okunabilir biçime çevirir^[3]

Biçim	Açıklama
<code>-o wide</code>	Ek sütunlar; Pod'larda node adı
<code>-o yaml</code>	YAML biçiminde tam API nesnesi
<code>-o json</code>	JSON biçiminde tam API nesnesi
<code>-o name</code>	Yalnızca nesne adı; script çıktısı sabit

```
kubectl get pods -o wide  
kubectl get pod nginx -o yaml  
kubectl get pods -o name
```



custom-columns ve JSONPath

- `-o custom-columns` seçili alanlardan tablo üretir; alanlar JSONPath ifadeleriyle belirtilir^[4]
- JSONPath ifadeleri `{...}` içinde yazılır; `{@}` geçerli nesneyi, `range` / `end` listeleri dolaşır

```
kubectl get nodes -o custom-columns='NAME:.metadata.name'  
kubectl get pods -o jsonpath='{.items[*].metadata.name}'  
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{end}'
```

- Kubernetes JSONPath'i, standart JSONPath'e ek olarak `range`, `end` ve `?()` filtrelerini destekler



Özet

- **Öğrenme ortamı:** kind (Docker tabanlı) veya minikube (tek node) ile ilk cluster kurulur
- **Bağlantı:** kubectl, kubeconfig ve context ile cluster'a erişilir
- **Günlük işlemler:** `get`, `describe`, `create`, `run`, `delete`, `logs`, `exec` ve çıktı biçimleri
- Sonraki adım: manifest okuryazarlığı ve declarative yönetim
→ **Kubernetes API ve Manifest Okuryazarlığı**

Kaynaklar





Kaynaklar

1. Öğrenme Ortamı: kubernetes.io/docs/setup/learning-environment/
2. kubectl Referansı: kubernetes.io/docs/reference/kubectl/
3. kubectl Hızlı Referans: kubernetes.io/docs/reference/kubectl/quick-reference/
4. JSONPath Desteği: kubernetes.io/docs/reference/kubectl/jsonpath/
5. kubeconfig ile Cluster Erişimi: kubernetes.io/docs/concepts/configuration/organize-cluster-access-kubeconfig/
6. kubectl Kurulumu (macOS): kubernetes.io/docs/tasks/tools/install-kubectl-macos/
7. kind Quick Start: kind.sigs.k8s.io/docs/user/quick-start/
8. minikube Başlangıç: minikube.sigs.k8s.io/docs/start/